ASSIGNMENT 5 - CO5

PART A - DESIGN

TASK A3 - EVALUATE VIRTUALISATION

5 Marks | Bloom's Level: K5 - Evaluate

Case Study and Design Objective

The university result portal is a Python web application with a database. It serves a few hundred users on ordinary days, but 150,000 students log in within two hours on result day. The current deployment uses one physical Linux server and crashes at peak. The university has a small private server room and an account with a public cloud provider.

Virtualisation divides physical resources into guest environments, consolidates services and makes maintenance/recovery more manageable. It does not create extra CPU, RAM or storage. Host capacity must be reserved and the design load-tested against the actual peak request profile.

1. A3.1 - Hypervisors (3 Marks)

A hypervisor, or virtual machine monitor (VMM), creates and manages VMs by presenting virtual CPU, memory, storage and network devices to guest operating systems and mediating their access to physical hardware.

Type	Architecture	Examples from the question	Evaluation
Type-1 Bare-metal	Runs directly on physical hardware. A management partition may provide drivers/administration, but the hypervisor is the hardware-virtualisation layer.	KVM, VMware ESXi, Xen, Hyper-V.	Server/datacentre design; no general-purpose desktop OS as the first layer.
Type-2 Hosted	Runs as an application on a host OS. The host OS controls physical devices and schedules the hypervisor process.	VirtualBox, VMware Workstation.	Convenient for desktop development, teaching and testing; adds host-OS scheduling and memory use.

KVM is listed in the assignment as Type-1. It uses Linux kernel modules (`kvm.ko` plus a processor-specific module) for core virtualisation, while QEMU supplies the user-space machine/device model. It is integrated with the Linux host kernel rather than being a desktop application. [1] Hyper-V also illustrates that a Type-1 design can include a root/management partition that owns physical I/O devices and provides virtual-device services to guests. [2]

2. Full Virtualisation and Para-Virtualisation

Concept	Operation	Benefit / limitation
Full virtualisation	The hypervisor presents virtual hardware so an unmodified guest OS can run. Hardware assistance and virtual-device emulation allow the guest to execute as a separate machine.	Broad guest compatibility; privileged operations and virtual-device I/O still need handling and can add overhead.
Para-virtualisation	The guest OS or drivers are aware of virtualisation and use hypercalls or hypervisor-aware interfaces rather than treating every operation as physical hardware access.	Can reduce selected costs, especially I/O and clock/timer operations; requires compatible guest support or drivers.

The methods can coexist: a guest can use hardware-assisted full virtualisation for CPU execution while using paravirtualised VirtIO devices for storage and network. VirtualBox also exposes guest-aware paravirtualisation providers. [3]

3. Hardware Support - Intel VT-x and AMD-V

Intel VT-x and AMD-V are x86 processor extensions that assist the hypervisor in running guests and regaining control for sensitive operations. The CPU must support the extension and it must be enabled in BIOS/UEFI. A visible `vmx` or `svm` flag is useful evidence, but usable virtualisation also depends on firmware, kernel modules, device permissions and any outer-hypervisor policy. [2][3]

Second-level address translation (Intel EPT / AMD NPT) reduces guest-to-host memory translation cost. IOMMU support can assist device assignment and DMA isolation. Check the selected host rather than assuming support from the CPU brand.

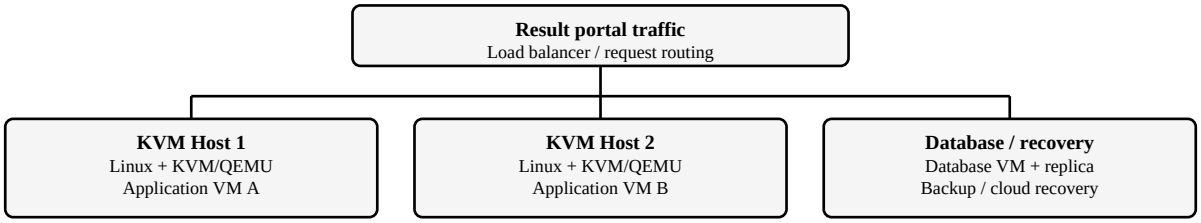
4. A3.2 - Benefits and Costs of Virtual Machines (2 Marks)

Benefit / cost	Value for the university	Condition / limitation
Isolation	Application and database run in separate guest OS instances, limiting ordinary process and OS interference.	Not absolute: hypervisor flaws, shared hardware, management compromise and network errors remain risks.
Consolidation	Several services share fewer physical servers, improving utilisation and reducing duplicated equipment.	Noisy-neighbour contention can occur; monitor CPU, RAM pressure, disk IOPS and network.
Snapshots	Capture VM state before patching/configuration changes and roll back if an update fails.	Point-in-time rollback, not independent backup; snapshot chains consume storage.
Live migration	Move a running VM between compatible hosts for maintenance or load balancing with limited interruption.	Needs compatible CPU/hypervisor/network and accessible shared storage; not automatic disaster recovery.
CPU / memory	Every guest has OS/application overhead; host OS and VMM/management processes also consume resources.	Overcommitment can cause latency and prevent restart after host failure.
Storage / licences	Virtual disks, snapshots, replicas and backups consume capacity and IOPS. Hypervisor support, guest OS and database licences may apply.	KVM avoids a per-hypervisor licence fee, but hardware, guest software, support and operations are not free.

A snapshot is useful for controlled rollback, but database-consistent backups must be maintained independently. Live migration requires compatible host CPUs, accessible shared storage and suitable networking; time and pause depend on VM memory and workload. [4]

5. Recommended Virtualisation Design

Use Type-1 KVM-based virtualisation on two or more compatible hosts. Separate the stateless application tier from the database. Place identical application VMs on different hosts behind a load balancer, and use database replication plus independent backups. Host-failure recovery requires tested restart/failover capacity; do not rely on snapshots alone.



Separate management, application and database networks

Illustrative: VM allocations and failover must be tested.

VM role	Illustrative starting allocation	Purpose
Application VM A	4 vCPU, 8 GiB RAM, 80 GiB disk	Host 1; application and web workers.
Application VM B	4 vCPU, 8 GiB RAM, 80 GiB disk	Host 2; identical app instance behind load balancer.
Database VM	8 vCPU, 16 GiB RAM, 200 GiB disk	Database workload on low-latency storage.
Management / monitoring	2 vCPU, 4 GiB RAM, 60 GiB disk	Monitoring and administration; restricted access.

These allocations are examples, not final sizing. The paper does not provide host CPU/RAM, storage IOPS, network speed, database size or peak request rate. Load testing must determine host count and VM allocations. Each host needs spare capacity to restart critical VMs after failure, or the university needs a tested cloud recovery plan.

5.1 Host memory budget

Physical RAM ≥ host OS + hypervisor/management + sum of guest RAM + cache + failure/headroom reserve.

The example guests allocate 8 + 8 + 16 + 4 = 36 GiB. The host needs additional memory for Linux services, QEMU/libvirt, caches and failover headroom. A two-host cluster must not allocate all RAM with no reserve; one host failure must not leave critical guests unable to restart.

6. Alternative Concept - Type-2 VirtualBox Instead of Type-1 KVM

VirtualBox runs as a program on a general-purpose host OS. The host OS manages physical CPU, memory, disk and network devices while VirtualBox processes manage guests. VirtualBox supports VT-x/AMD-V and guest-aware interfaces, but the hosted model adds host-OS scheduling, memory and device-driver layers. [3]

Evaluation	Type-1 KVM	Type-2 VirtualBox
CPU-bound	Hardware-assisted guest execution; host Linux schedules vCPUs. KVM/QEMU and host scheduling still add overhead.	Hardware assistance applies, but vCPU threads also use the host scheduler. Background services can compete for CPU and increase latency variability.

Evaluation	Type-1 KVM	Type-2 VirtualBox
I/O-bound	Linux host I/O stack; VirtIO/paravirtualised devices can reduce virtual I/O overhead. Host I/O still occurs.	Virtual disks often reside as files on the host filesystem; requests pass through VMM and host OS drivers. Paravirtualised drivers help but hosted layers remain.
Host memory	Lean Linux server host and QEMU/libvirt still consume RAM.	Full host OS/background services, VirtualBox/VMM processes and guests consume RAM.
Production fit	Better fit for managed server clusters with resource control and configured migration/HA.	Best for developer desktops, demos and labs; less suitable as the production portal platform.

This is a relative evaluation, not a claim that VirtualBox cannot run server guests. It can run headlessly and uses hardware virtualisation/paravirtualisation, but host-OS dependence makes it less attractive for a portal with sharp peak demand and recovery requirements. Oracle documents VirtualBox's host processes, VMM and hardware acceleration. [3]

6.1 CPU, I/O and memory overhead

For CPU-bound work, both hypervisors use hardware assistance; Type-2 adds host-OS scheduling and resource contention. For I/O-bound work, both translate virtual-device operations, but VirtualBox's hosted disk/network path adds another OS layer. KVM can use VirtIO and a lean Linux host. Actual percentages depend on CPU, guest drivers, virtual-disk format/cache, vCPU count and host load; none should be claimed without controlled benchmarks.

Both designs need host OS RAM + VMM/management + guest allocations + cache + reserve. VirtualBox keeps a general-purpose host OS and its background applications resident. KVM also needs Linux, but it can be installed without a desktop environment. Thus the comparison is a full desktop-host baseline versus a lean server-host baseline, not zero host memory for KVM. The illustrative VM set uses 36 GiB guest RAM before host overhead.

6.2 Production suitability

VirtualBox suits development, demonstrations, compatibility testing and small labs. For the result portal, Type-1 KVM is preferred for server-oriented resource management, separate application/database roles and the ability to build a cluster with tested restart/migration procedures. High availability still requires compatible hosts, shared or replicated storage, network redundancy and database recovery.

7. Complexity Analysis and Measurement

Virtualisation does not change the portal algorithm. If an application operation is O(n) or O(n log n) on bare metal, its algorithmic order normally remains the same in either hypervisor. Virtualisation adds runtime overhead rather than changing application Big-O complexity.

Measure	Type-1 KVM	Type-2 VirtualBox
Execution time	$T_{KVM}(n) = T_{guest}(n) + H_{KVM}(n)$, for hypervisor, device and host-scheduling overhead.	$T_{VBox}(n) = T_{guest}(n) + H_{VBox}(n) + H_{hostOS}(n)$, including extra host-layer work.
Asymptotic order	Usually same O(f(n)); overhead is a constant-factor question to measure.	Also same O(f(n)); host-layer work may increase constants and latency variation.
Memory space	$O(M_{host} + M_{VMM} + \Sigma M_{guest} + M_{cache} + M_{reserve})$.	$O(M_{hostOS} + M_{VBox} + \Sigma M_{guest} + M_{cache} + M_{reserve})$.
VM fleet	Tracking V VM definitions/resources requires at least O(V); templates reduce repeated manual work.	Also O(V); workstation-specific settings can make consistent fleet management harder.

The overhead terms depend on VM exits, virtual-device operations, scheduling events and cache misses. They cannot be assigned a numeric value without measurements. Compare identical guest images, vCPU/RAM, storage and workloads; record CPU throughput, IOPS, latency, host RAM and startup time.

7.1 Evidence procedure

The included read-only Bash script checks CPU virtualisation flags, `/dev/kvm`, KVM modules, QEMU/libvirt/VirtualBox tools, host memory and storage. Run it on the actual Linux host:

```
chmod +x virtualization_environment_check.sh
./virtualization_environment_check.sh

# Output: a3_virtualization_host_check.txt
```

The worksheet gives CPU tests with `sysbench`, random-read I/O tests with `fio` on a disposable virtual disk, host/guest RAM observations and VM cold-start timing. Repeat at least three times. Never run `fio` against a raw device or important data volume. WSL/nested VMs may hide flags and `/dev/kvm`; record this limitation.

8. Conclusion and Recommendation

Type-1 hypervisors such as KVM, ESXi, Xen and Hyper-V are the stronger fit for production server infrastructure. Type-2 hypervisors such as VirtualBox and VMware Workstation are useful for desktop development and testing. Full virtualisation supports unmodified guests; para-virtualised interfaces can improve selected I/O paths. Intel VT-x and AMD-V provide hardware assistance and must be enabled in firmware.

For the university portal, use KVM with separate application/database VMs, identical application instances across hosts, resource reservations, redundant storage, independent database backups and tested recovery. Reserve VirtualBox for development and demonstrations. Confirm final VM sizes and overhead with controlled measurements.

9. Technical References

[1] Linux KVM Project, *Kernel Virtual Machine* - KVM modules, full virtualisation and Intel VT/AMD-V. https://www.linux-kvm.org/page/Main_Page

[2] Microsoft Learn, *Hyper-V Architecture* - Type-1 hypervisor, root/child partitions, hardware virtualisation and VMBus I/O. https://learn.microsoft.com/en-us/windows-server/virtualization/hyper-v/architecture

[3] Oracle, *VirtualBox 7.0 Technical Background* - host processes, VT-x/AMD-V, emulated devices and paravirtualisation. https://docs.oracle.com/en/virtualization/virtualbox/7.0/user/TechnicalBackground.html

[4] libvirt, *Virtual Machine Lifecycle* - snapshots, live migration and shared-storage/CPU/network compatibility. https://libvirt.gitlab.io/libvirt-wiki/VM_lifecycle.html

Source alignment: Assignment 5 - CO5, Part A - Design, Task A3 - Evaluate Virtualisation, 5 Marks, page 44. General task requirements appear on page 42.